Computer Systems and Networks

Prof. Ramachandran, Prof. Forsyth

Project 4 - Process Scheduling

Due: **April 17th 2026**

1 Introduction

In this project, you will implement a multiprocessor operating system simulator using a threading library for Linux called `pthreads`. The framework for the multiprocessor OS simulator is nearly complete, but missing one critical component: the process scheduler! Your task is to implement the process scheduler and three different scheduling algorithms.

2 Grading

This project is not split into any checkpoints and has just the final due date. The grading for this project will be done based on the statistics of your scheduling algorithm when simulating a series of different processes that each take some amount of time on the CPU and on I/O. There will also be a few short answer questions that involves analyzing some of the stats from your algorithms, which you will write in a text file and submit to Gradescope along with your code. The grading distribution is as follows:

Test Case	**Points Awarded**
Autograder	
Code Submitted Correctly	2 points
Round Robin Code	9 points
FCFS Code	9 points
SRTF Code	9 points
Priority Code	9 points
Round Robin Race Conditions	3 points
FCFS Race Conditions	3 points
SRTF Race Conditions	3 points
Priority Race Conditions	3 points
Manual Grading	
Code Locks Correctly	25 points
Short Answer: FCFS Scheduler	5 points
Short Answer: Round Robin	5 points
Short Answer: Preemptive Priority Scheduler	5 points
Short Answer: Priority Inversion Problem	10 points

Table 1: Grade Breakdown

3 Overview

The simulated operating system supports only one thread per process making it similar to the systems that we discussed in Chapter 6. However, the simulator itself will use a thread to represent each of the CPUs in the simulated hardware. This means that the CPUs in the simulator will appear to operate concurrently.

Note: Multiple CPU cores need to be enabled for this to work correctly. The CS 2200 Docker container should already be configured to run with 4 cores. Please see a TA if you are running into any problems.

We have provided you with source files that constitute the framework for your simulator. You will only need to modify `answers.txt` and `student.c`. However, just because you are only modifying two files doesn't mean that you should ignore the other ones.

We have provided you the following files:

1. `os-sim.c` - Code for the operating system simulator which calls your CPU scheduler.
2. `os-sim.h` - Header file for the simulator.

3. `process.c` - Descriptions of the simulated processes.
4. `process.h` - Header file for the process data.
5. `student.c` - This file contains stub functions for your CPU scheduler.
6. `student.h` - Header file for your code to interface with the OS simulator. Also contains the ready queue struct definition.
7. `answers.txt` - This is a text file that you should use to write your answers to the questions listed throughout the PDF.

Reminder: The only files that you need to edit are `student.c` and `answers.txt`. If you edit any other files, your code may have unexpected outputs!

3.1 Scheduling Algorithms

For your simulator, you will implement the following four CPU scheduling algorithms:

1. **First Come, First Serve (FCFS)** - Runnable processes are kept in a ready queue. FCFS is non-preemptive; once a process begins running on a CPU, it will continue running until it either completes or blocks for I/O.
2. **Round-Robin** - Similar to FCFS, except preemptive. Each process is assigned a timeslice when it is scheduled. At the end of the timeslice, if the process is still running, the process is preempted, and moved to the tail of the ready queue.
3. **Preemptive Priority Scheduling** - Processes with higher priority get to run first and processes with lower priority get preempted for a process with higher priority.
4. **Shortest Remaining Time First** - The process with the shortest time remaining is chosen. Time remaining includes all future CPU and IO bursts. Currently scheduled processes are preempted if a process with a smaller time remaining is ready to be scheduled.

3.2 Process States

In our OS simulation, there are five possible states for a process. These states are listed in the `process_state_t` enum in `os-sim.h`:

1. `NEW` - The process is being created, and has not yet begun executing.
2. `READY` - The process is ready to execute, and is waiting to be scheduled on a CPU.
3. `RUNNING` - The process is currently executing on a CPU.
4. `WAITING` - The process has temporarily stopped executing, and is waiting for an I/O request to complete.
5. `TERMINATED` - The process has completed.

There is a field named `state` in the PCB, which must be updated with the current state of the process. **The simulator will use this field to collect statistics.**

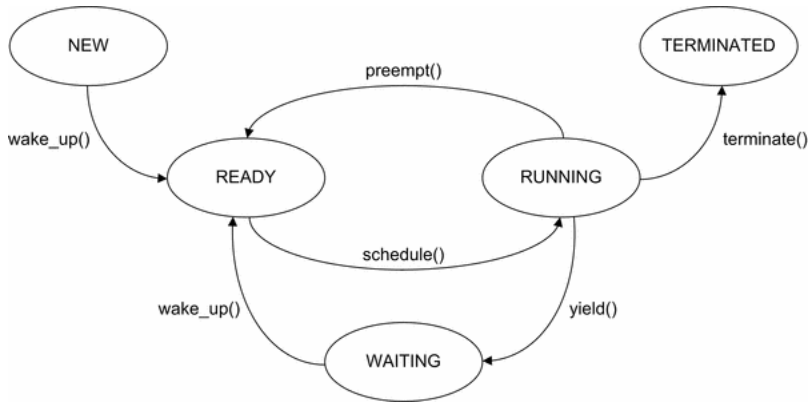

Figure 1: Process States

3.3 The Ready Queue

On most systems, there are a large number of processes that need to share the resources of a small number of CPUs. When there are more processes ready to execute than CPUs, processes must wait in the `READY` state until a CPU becomes available. To keep track of the processes waiting to execute, we keep a ready queue of the processes in the `READY` state.

Since the ready queue is accessed by multiple processors, which may add and remove processes from the ready queue, the ready queue must be protected by some form of synchronization. For this project, you will use a mutex lock that we have provided called `ready_mutex`.

3.4 Scheduling Processes

`schedule()` is the core function of the CPU scheduler. It is invoked whenever a CPU becomes available for running a process. `schedule()` must search the ready queue, select a runnable process, and call the `context_switch()` function to switch the process onto the CPU.

Note that in a multiprocessor environment, we cannot mandate that the currently running process be at the head of the ready queue. There is an array (one entry for each CPU) that will hold the pointer to the PCB currently running on that CPU.

There is a special process, the "idle" process, which is scheduled whenever there are no processes in the `READY` state. This process simply waits for something new to be added to the ready queue and then calls `schedule()`.

3.5 CPU Scheduler Invocation

There are five events which will cause the simulator to invoke `schedule()`:

1. `yield()` - A process completes its CPU operations and yields the processor to perform an I/O request.
2. `wake_up()` - A process that previously yielded completes its I/O request, and is ready to perform CPU operations. `wake_up()` is also called when a process in the NEW state becomes runnable.
3. `on_preempt()` - When using a preemptive scheduling algorithm, a CPU-bound process may be preempted before it completes its CPU operations.
4. `terminate()` - A process exits or is killed.
5. `idle()` - Waits for a new process to be added to the ready queue. `idle()` contains the code that gets executed by the idle process. In the real world, the idle process puts the processor in a low-power mode

and waits. For our OS simulation, you will use a pthread condition variable to block the thread until a process enters the ready queue.

3.6 The Simulator

We will use pthreads to simulate an operating system on a multiprocessor computer. We will use one thread per CPU and one thread as a 'supervisor' for our simulation. The supervisor thread will spawn new processes (as if a user started a process). The CPU threads will simulate the currently-running processes on each CPU, and the supervisor thread will print output.

Since the code you write will be called from multiple threads, the CPU scheduler you write must be thread-safe! This means that all data structures you use, including your ready queue, must be protected using mutexes.

The number of CPUs is specified as a command-line parameter to the simulator. For this project, you will be performing experiments with 1, 2, and 4 CPU simulations.

Also, for demonstration purposes, the simulator executes much slower than a real system would. In the real world, a CPU burst might range from one to a few hundred milliseconds, whereas in this project, they will normally be on the scale of seconds.

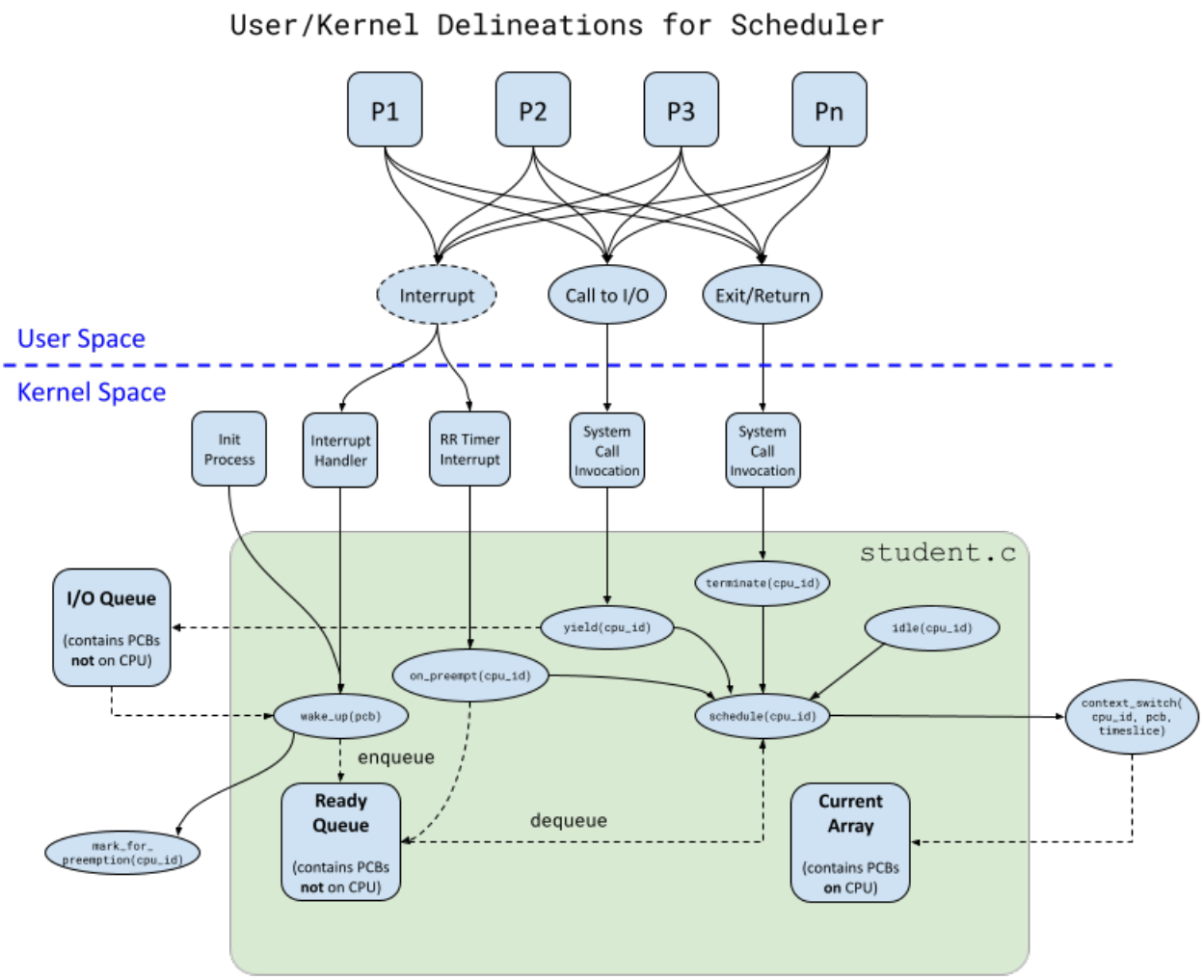

Figure 2: System overview

The above diagram demonstrates the delineation between user and kernel space in the simulator. All the code for the simulator is in kernel space since the user processes themselves are just simulated. Your code will be everything inside the green bubble. The flow of various functions and data structures including the ready queue, I/O queue, and the array of currently running PCBs is also depicted in the diagram.

Compile and run the simulator with `./os-sim 2`. After a few seconds, hit Control-C to exit. You will see the output below:

```
Time  Ru Re Wa      CPU 0    CPU 1        < I/O Queue <

===== == == ==      ======== ========     =============

0.0   0  0  0       (IDLE)   (IDLE)        < <

0.1   0  0  0       (IDLE)   (IDLE)        < <

0.2   0  0  0       (IDLE)   (IDLE)        < <

0.3   0  0  0       (IDLE)   (IDLE)        < <

0.4   0  0  0       (IDLE)   (IDLE)        < <

0.5   0  0  0       (IDLE)   (IDLE)        < <

0.6   0  0  0       (IDLE)   (IDLE)        < <

0.7   0  0  0       (IDLE)   (IDLE)        < <

0.8   0  0  0       (IDLE)   (IDLE)        < <

0.9   0  0  0       (IDLE)   (IDLE)        < <

1.0   0  0  0       (IDLE)   (IDLE)        < <

......
```

Figure 3: Sample Output

The simulator generates a Gantt Chart, showing the current state of the OS at every 100ms interval. The leftmost column shows the current time, in seconds. The next three columns show the number of Running, Ready, and Waiting processes, respectively. The next two columns show the process currently running on each CPU. The rightmost column shows the processes which are currently in the I/O queue, with the head of the queue on the left and the tail of the queue on the right.

As you can see, nothing is executing. This is because we have no CPU scheduler to select processes to execute! Once you complete Problem 1 and implement a basic FCFS scheduler, you will see the processes executing on the CPUs.

4 Problem 0: The Ready Queue

We have provided simple implementations of `queue_t`, `enqueue()`, `dequeue()`, and `is_empty()` in `student.c`. The struct you have to implement will serve as your ready queue, and you should be using these helper functions to add and remove processes from the ready queue in the problems to follow.

4.1 Provided Queue

- The queue is backed by a linked list with each PCB acting as a node. There is a field in the PCB, `next`, which you may use to build linked lists of PCBs.
- `enqueue()` will add a process to the ready queue at the appropriate location.
- `dequeue()` will remove a process from the ready queue at the appropriate location and return a pointer to that process.
- You may implement process ordering in either `enqueue()` or `dequeue()` — both approaches are acceptable.
- **NOTE: When using the ready queue helper functions in the following problems, make sure to call them in a thread-safe manner.** Read up on how to use mutex locks and lock/unlock the mutex for the ready queue when you call these functions. **You will need to modify the `enqueue()` and `dequeue()` functions to support different scheduling algorithms in Problem 3.**

5 Problem 1: FCFS Scheduler

Implement the CPU scheduler using the FCFS scheduling algorithm. You may do this however you like, however, we suggest the following:

- **Implement the `yield()`, `wake_up()`, and `terminate()` handlers. in `student.c`.**

  Checkout the hints in section 3.1, and note that `on_preempt()` is not necessary for this stage of the project.

- **Implement `idle()`.**

  `idle()` must wait on a condition variable that is signalled whenever a process is added to the ready queue.

- **Implement `schedule()`.**

  `schedule()` should extract the first process in the ready queue, update the PCB of the selected CPU, and call `context_switch()` to begin executing the new process. If there are no runnable processes, `schedule()` should call `context_switch()` with a null PCB to execute the idle process.

5.1 Hints

- Make sure your helper functions are thread-safe when adding and removing processes from the ready queue!
- The `current[]` array keeps track of the process currently running on each CPU. Be sure to update the `state` field of the PCB in all the functions above. The library will read this field to generate the RUNNING, READY, and WAITING columns and calculate the statistics at the end of the simulation.
  - Since the `current` array is accessed by multiple CPU threads, it must be protected by a mutex. `current_mutex` has been provided for you.
- Four of the five entry points into the scheduler (`idle()`, `yield()`, `terminate()`, and `on_preempt()`) should cause a new process to be scheduled on the CPU. In your handlers, be sure to call `schedule()`, which will select a runnable process and call `context_switch()`. When your four entry points return, the library will simulate the execution of the process selected by `context_switch()`.
- `context_switch()` takes a timeslice parameter, which is used for preemptive scheduling algorithms. Since FCFS is non-preemptive, use -1 for this parameter to give the process an infinite timeslice.

6 Problem 2: Round-Robin Scheduler

Add Round-Robin scheduling functionality to your code.

For this project, timeslices are measured in tenths of seconds. E.g.:

```
./os-sim <# CPUs> -r 5
```

should run a Round-Robin scheduler with timeslices of 500 ms. While:

```
./os-sim <# of CPUs>
```

should continue to run a FCFS scheduler.

Since Round-Robin doesn't take into account arrival time, it requires different logic from FCFS in the `dequeue()` function.

Implement `on_preempt()`.

`on_preempt()` should place the currently running process back in the ready queue, and call schedule() to select a new runnable process.

To specify a timeslice when scheduling a process, use the timeslice parameter of `context_switch()`. The simulator will simulate a timer interrupt to preempt the process and call your `on_preempt()` handler if the process executes on the CPU for the length of the timeslice without terminating or yielding for I/O.

7 Problem 3: Preemptive Priority Scheduling

Add Priority scheduling to your code. Alter the provided `enqueue()` and/or `dequeue()` to support this algorithm.

To complete this, you may also need to modify the `wake_up()` function. In order to do this, loop through all the CPUs and find the lowest priority process.

7.1 Preemption

Recall that Priority is a preemptive algorithm, but, unlike Round Robin, it doesn't use a timeslice. Instead, lower-priority processes are preempted when higher-priority processes are ready to run. Determine which of the entry points from section 3 need to be modified to support preemption. Once you have determined which CPU to preempt, you must use the `mark_for_preemption()` method (NOT `on_preempt()`) to return its currently running process to the ready queue.

`mark_for_preemption(uint cpu_id)`

This function will help you implement the priority scheduling. Once you have found the lowest priority process and what CPU it resides on, use this API to mark that CPU for preemption.

8 Problem 4: Shortest Remaining Time First

Add Shortest Remaining Time First scheduling to your code. Alter the provided `enqueue()` and/or `dequeue()` to support scheduling based on total time remaining.

Under SRTF scheduling, the process with the smallest amount of time remaining is chosen to be scheduled. This process is allowed time on the CPU until its burst finishes. At any given time a new process can enter the ready queue. If this process' time remaining is smaller than the remaining time for a currently scheduled process, that process is preempted and the new process is scheduled.

You should find shortest remaining time over every process in the ready queue and schedule the process with the smallest amount of time remaining. **This means your ready queue does not have to stay in order by time remaining.** Just like Priority, we look at each process in the ready queue to determine which one to dequeue.

SRTF is a preemptive algorithm without a timeslice, so, like Priority, you will need to update one of the entry points you wrote in section 3 to support preemption.

9 Problem 5: Short Answers

Please write your answers to the following questions in `answers.txt`.

9.1 FIFO Scheduler

Run your OS simulation with 1, 2, and 4 CPUs. Compare the total execution time of each. Is there a linear relationship between the number of CPUs and total execution time? Why or why not? Keep in mind that the execution time refers to the simulated execution time.

9.2 Round-Robin Scheduler

Run your Round-Robin scheduler with timeslices of 800ms, 600ms, 400ms, and 200ms. Use only one CPU for your tests. Compare the statistics at the end of the simulation. Is there a relationship between the total waiting time and timeslice length? If so, what is it? In contrast, in a real OS, the shortest timeslice possible is usually not the best choice. Why not?

9.3 Preemptive Priority Scheduler

Priority schedulers can sometimes lead to starvation among processes with lower priority. What is a way that operating systems can mitigate starvation in a priority scheduler?

9.4 Priority Inversion

Consider a non-preemptive priority scheduler. Suppose you have a high-priority process (P1) that wants to display a window on the monitor. But, the window manager is a process with low priority and will be placed at the end of the ready queue. While it is waiting to be scheduled, new medium-priority processes are likely to come in and starve the window manager process. The starvation of the window manager will also mean the starvation of P1 (the process with high priority), since P1 is waiting for the window manager to finish running.

If we want to keep our non-preemptive priority scheduler, what edits can we make to our scheduler to ensure that the P1 can finish its execution before any of the medium priority processes finish their execution? Explain in detail the changes you would make.

10 The Gradescope Environment

You will be submitting files to Gradescope, where they will be tested in a VM setup that runs through Gradescope. The specifications of this VM are that it runs `Ubuntu 22.04 LTS (64-bit)` and `gcc 13.2.0`, and so we expect that your files can run in such a setup. This means that *when you are running your project locally*, you will want to ensure you are using a VM/some setup that runs `Ubuntu 22.04 LTS (64-bit)` and `gcc 13.2.0`; this way, you can ensure that what occurs locally is what will occur when you submit to Gradescope.

IMPORTANT: Since we are dealing with different threads of execution, the result of each run in the simulation will be different. As a result, our gradescope autograder will accept a range of results for your simulation. In past semesters, we found that the range will start to **increase** the more computations you do in your enqueue/dequeue functions. If you find that you aren't passing the autograder but you believe that your code is right, it is likely that you need to trim down your enqueue/dequeue functions. We are planning to start our autograder with a tighter acceptable range, and if required, we will increase it.

11 Deliverables

NOTE: You need to upload `student.c`, and `answers.txt` to Gradescope, and an autograder will run to check if your scheduler is working. The autograder might take a couple of minutes to run. Remember to upload `student.c`, and `answers.txt` for every submission as your last submission would be one we will grade.

Keep your answers detailed enough to cover the question, including support from simulator results if appropriate. Don't write a book; but if you're not sure about an answer, give us more information.

12 How to Run / Debug Your Code – Debugging deadlocks and synchronization errors

12.1 Running

We have provided a Makefile that will run gcc for you. To compile your code with no optimizations (which you should do while developing, it will make debugging easier) and test with the FCFS algorithm and one CPU, run:

```
$ make debug
$./os-sim 1
```

To run the other algorithms, run with the flags you implemented for round robin and priority. Remember that round robin requires you to enter a time slice.

In case you encounter difficulties with Project 4 and are uncertain about the direction to take, various resources are available to assist you.

12.2 GDB

Let us investigate how to debug deadlocks through a basic example:

```
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1;
pthread_mutex_t lock2;

void *thread1(void *data) {
        pthread_mutex_lock(&lock1);
        sleep(1);
        pthread_mutex_lock(&lock2);
        pthread_mutex_unlock(&lock2);
        pthread_mutex_unlock(&lock1);
}

void *thread2(void *data) {
        pthread_mutex_lock(&lock2);
        sleep(1);
        pthread_mutex_lock(&lock1);
        pthread_mutex_unlock(&lock1);
        pthread_mutex_unlock(&lock2);
}

int main() {
        pthread_t t1, t2;
        pthread_mutex_init(&lock1, NULL);
        pthread_mutex_init(&lock2, NULL);

        pthread_create(&t1, NULL, thread1, NULL);
        pthread_create(&t2, NULL, thread2, NULL);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
}
```

Following the execution and compilation of the code, it appears to become unresponsive. To investigate the root cause of this issue, it is recommended to utilize the GNU Debugger (gdb) to identify the problem. To set up the above program in gdb:

```
$ gcc -g <filename>.c -o threadTest
$ gdb <filename>.c
$ file threadTest
```

Now let's run the program in gdb:

```
(gdb) r
Starting program: /cs2200/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[New Thread 0x7ffff7d8b640 (LWP 40)]
[New Thread 0x7ffff758a640 (LWP 41)]
```

As anticipated, the program continues to remain unresponsive when run within the gdb environment. To interrupt the program, press Ctrl + c. To analyze the various threads associated with the program, utilize the ***info threads*** command within gdb, which provides detailed information regarding each active thread:

```
(gdb) info threads
  Id   Target Id                                       Frame
* 1    Thread 0x7ffff7d8c740 (LWP 160) "threadTest" __futex_abstimed_wait_common64 (private=128, cancel=true,
    abstime=0x0, op=265, expected=161, futex_word=0x7ffff7d8b910) at ./nptl/futex-internal.c:57
  2    Thread 0x7ffff7d8b640 (LWP 161) "threadTest" futex_wait (private=0, expected=2,
    futex_word=0x555555558080 <lock2>) at ../sysdeps/nptl/futex-internal.h:146
  3    Thread 0x7ffff758a640 (LWP 162) "threadTest" futex_wait (private=0, expected=2,
    futex_word=0x555555558040 <lock1>) at ../sysdeps/nptl/futex-internal.h:146
```

Upon examining the running threads, we can observe that threads 2 and 3 are currently in the ***futex_wait*** state. ***futex_wait*** is a linux kernel sys call that shows that a thread is waiting to acquire a mutex or some other synchronization primitive. Let's take a deeper look at what is happening.

To obtain the backtrace of these threads, we can run ***thread apply all backtrace***, abbreviated as ***t a a bt***

```
(gdb) thread apply all backtrace

Thread 3 (Thread 0x7ffff758a640 (LWP 162) "threadTest"):
#0  futex_wait (private=0, expected=2, futex_word=0x555555558040 <lock1>) at ../sysdeps/nptl/futex-internal.h:146
#1  __GI___lll_lock_wait (futex=futex@entry=0x555555558040 <lock1>, private=0) at ./nptl/lowlevellock.c:49
#2  0x00007ffff7e27002 in lll_mutex_lock_optimized (mutex=0x555555558040 <lock1>) at ./nptl/pthread_mutex_lock.c:48
#3  ___pthread_mutex_lock (mutex=0x555555558040 <lock1>) at ./nptl/pthread_mutex_lock.c:93
#4  0x00005555555529a in thread2 (data=0x0) at threadingTest.c:19
#5  0x00007ffff7e23ac3 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:442
#6  0x00007ffff7eb5850 in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:81

Thread 2 (Thread 0x7ffff7d8b640 (LWP 161) "threadTest"):
#0  futex_wait (private=0, expected=2, futex_word=0x555555558080 <lock2>) at ../sysdeps/nptl/futex-internal.h:146
#1  __GI___lll_lock_wait (futex=futex@entry=0x555555558080 <lock2>, private=0) at ./nptl/lowlevellock.c:49
#2  0x00007ffff7e27002 in lll_mutex_lock_optimized (mutex=0x555555558080 <lock2>) at ./nptl/pthread_mutex_lock.c:48
#3  ___pthread_mutex_lock (mutex=0x555555558080 <lock2>) at ./nptl/pthread_mutex_lock.c:93
#4  0x0000555555555241 in thread1 (data=0x0) at threadingTest.c:11
#5  0x00007ffff7e23ac3 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:442
#6  0x00007ffff7eb5850 in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:81

Thread 1 (Thread 0x7ffff7d8c740 (LWP 160) "threadTest"):
#0  __futex_abstimed_wait_common64 (private=128, cancel=true, abstime=0x0, op=265, expected=161, futex_word=0x7ffff7d8b9
10) at ./nptl/futex-internal.c:57
#1  __futex_abstimed_wait_common (cancel=true, private=128, abstime=0x0, clockid=0, expected=161, futex_word=0x7ffff7d8b
910) at ./nptl/futex-internal.c:87
#2  __GI___futex_abstimed_wait_cancelable64 (futex_word=futex_word@entry=0x7ffff7d8b910, expected=161, clockid=clockid@e
ntry=0, abstime=abstime@entry=0x0, private=private@entry=128) at ./nptl/futex-internal.c:139
#3  0x00007ffff7e25624 in __pthread_clockjoin_ex (threadid=140737351562816, thread_return=0x0, clockid=0, abstime=0x0, b
lock=<optimized out>) at ./nptl/pthread_join_common.c:105
#4  0x0000555555555349 in main () at threadingTest.c:32
```

The backtrace command confirms that threads 2 and 3 are indeed stuck at the pthread_mutex_lock function (look at line numbers on #4 for each thread). To gain a more in-depth understanding of the specific thread's

state, we can utilize the gdb command ***thread [thread number]*** to switch to a particular thread and examine its current state. You can apply the same functions as above to this particular thread to get a better/more detailed view.

Once you have identified the problematic line, you can utilize gdb's features, such as printing values or switching stack frames, to investigate further and gain a better understanding of the issue at hand. Read the gdb thread documentation here for more information.

12.3 Valgrind (Helgrind or DRD)

What about issues when accessing a shared resource? Valgrind has some handy tools to help detect such problems.

12.3.1 Installation:

The current CS 2200 docker image does not come with valgrind installed. To install valgrind:

```
$ apt update
$ apt install valgrind
```

Follow the prompts in the terminal.

Note: Because of how docker works, you might have to run these commands every time you open your docker container.

Note: While you can complete the project without this tool, it is highly encouraged that you use this tool to help debug your program

12.3.2 :

Let's modify the code from the previous section:

```
#include <pthread.h>

pthread_mutex_t lock;
int shared;

void *thread1(void *data) {
        pthread_mutex_lock(&lock);
        shared = 1;
        pthread_mutex_unlock(&lock);
}

void *thread2(void *data) {
        shared = 2;
}

int main() {
        pthread_t t1, t2;
        pthread_mutex_init(&lock, NULL);
        pthread_create(&t1, NULL, thread1, NULL);
        pthread_create(&t2, NULL, thread2, NULL);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
}
```

You can run Helgrind with the command '***valgrind --tool=helgrind <./compiled_output>***'. This is the result:

```
==565==  Lock at 0x10C040 was first observed
==565==    at 0x4854BFE: pthread_mutex_init (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==565==    by 0x10926B: main (valgrindTest.c:18)
==565==  Address 0x10c040 is 0 bytes inside data symbol "lock"
==565==
==565== Possible data race during write of size 4 at 0x10C068 by thread #3
==565== Locks held: none
==565==    at 0x109230: thread2 (valgrindTest.c:13)
==565==    by 0x485396A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==565==    by 0x48FAAC2: start_thread (pthread_create.c:442)
==565==    by 0x498BA73: clone (clone.S:100)
==565==
==565== This conflicts with a previous write of size 4 by thread #2
==565== Locks held: 1, at address 0x10C040
==565==    at 0x109208: thread1 (valgrindTest.c:8)
==565==    by 0x485396A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==565==    by 0x48FAAC2: start_thread (pthread_create.c:442)
==565==    by 0x498BA73: clone (clone.S:100)
==565==  Address 0x10c068 is 0 bytes inside data symbol "shared"
```

Upon executing Valgrind's tool Helgrind, we can observe that it has successfully identified an issue within the program where thread2 is accessing a shared variable without acquiring a corresponding lock. Additionally, DRD (another tool within Valgrind) also provides comparable output, albeit with fewer error lines. It is essential to rectify these issues to ensure proper synchronization and avoid potential data race conditions. To compare, here is the same program run with DRD ('valgrind tool=drd <./compiled_output>'):

Valgrind and DRD are also able to debug other types of synchronization errors. You can read the documentation about Helgrind here and DRD here.

Credit to this video from an old class.

13 Docker Installation Instructions

For this project, you need to have Docker installed to be able to run your code locally on your computer. Below, we have provided instructions (same as on Canvas) on how to do so.

1. Install the latest version of Docker. (This is the same Docker app used in 2110, so you can skip this step if you already have Docker installed)
   - Windows
   - Mac
2. Run:

   ```
   $ docker pull systemsandnetworks/cs2200-students
   ```

3. Open the docker desktop application and let it run in the background
4. Download and unzip the docker.zip folder from Canvas Files
5. Type either of the following into your terminal (These commands must be run in the extracted docker folder):
   - Windows: `$ .\run.bat`
   - Mac & Linux: Run `$ chmod +x run.sh` and then `$ ./run.sh`

If step 5 works correctly, you should see something like this:

```
root@57a4f8b9fe5b:/cs2200
```

You can close the container by typing exit or pressing Ctrl+D. You can start the container again by running the same command as in step 4.

13.1 How the workspace folder works

The workspace folder is mounted to the container, which means that everything in the workspace folder is shared between your computer and the docker container. Whenever you spin up a container (by running `./run.sh` or `.\run.bat`), any changes made to files in the /cs2200/ folder inside the container will persist to your host file system and be visible in the workspace/ folder. You should put your homework and project files in this folder when using the container.

13.2 Opening additional terminals

To open additional terminals for the same docker session, use the attach script we gave you in a separate terminal. Execute it the same way as (step 4) the run.bat/run.sh script (including running chmod on it)